

Busca em profundidade p/ digrafos

Versão não recursiva

procedimento $P(v)$

$t \leftarrow t + 1; PE(v) \leftarrow t$

para todo vértice w em $N_{out}(v)$ faça

se $PE(w) = 0$

então { marcar vw como *aresta “azul” da floresta de profundidade T*
 $pai(w) \leftarrow v$
executar $P(w)$

senão { se $PS(w) = 0$
então marcar vw como *aresta “vermelha” de retorno*
senão se $PE(v) < PE(w)$ (*se v entrou antes de w na busca*)
então marcar vw como *aresta “amarela” de avanço*
senão marcar vw como *aresta “verde” cruzamento*

fim-para

$t \leftarrow t + 1; PS(v) \leftarrow t$

fim-do-procedimento

Código inicial

Busca em profundidade p/ digrafos

Versão não recursiva

procedimento $P(v)$

$t \leftarrow t + 1; PE(v) \leftarrow t$

para todo vértice w em $N_{out}(v)$ faça

se $PE(w) = 0$

então { marcar vw como *aresta “azul” da floresta de profundidade T*
 $pai(w) \leftarrow v$
executar $P(w)$

senão { se $PS(w) = 0$
então marcar vw como *aresta “vermelha” de retorno*

fim-para

$t \leftarrow t + 1; PS(v) \leftarrow t$

fim-do-procedimento

Olhando apenas arestas
pai-filho e de **retorno**

Busca em profundidade p/ digrafos

Versão não recursiva

procedimento $P(v)$

$PE(v) \leftarrow 1$

para todo vértice w em $N_{out}(v)$ faça

se $PE(w) = 0$

então { marcar vw como *aresta “azul” da floresta de profundidade T*
 $pai(w) \leftarrow v$
executar $P(w)$

senão { se $PS(w) = 0$
então marcar vw como *aresta “vermelha” de retorno*

fim-para

$PS(v) \leftarrow 1$

fim-do-procedimento

Deixando de marcar
o tempo

Busca em profundidade p/ digrafos

Versão não recursiva

procedimento $P(v)$

$PE(v) \leftarrow 1$

para todo vértice w em $N_{out}(v)$ faça

se $PE(w) = 0$

então

$\left\{ \begin{array}{l} \text{executar } P(w) \end{array} \right.$

se $PS(w) = 0$

então marcar vw como *aresta “vermelha” de retorno*

senão

fim-para

$PS(v) \leftarrow 1$

fim-do-procedimento

Deixando de marcar
o pai

Busca em profundidade p/ digrafos

Versão não recursiva

procedimento $P(v)$

$PE(v) \leftarrow 1$

para todo vértice w em $N_{out}(v)$ faça

se $PE(w) = 0$

então

executar $P(w)$

senão

se **w está na pilha**

então marcar vw como **aresta “vermelha” de retorno**

fim-para

fim-do-procedimento

Eliminando PS

Busca em profundidade p/ digrafos

Versão não recursiva

procedimento $P(v)$

$PE(v) \leftarrow 1$

para todo vértice w em $N_{out}(v)$ faça

se $PE(w) = 0$

então executar $P(w)$

senão se w está na pilha

então marcar vw como aresta “**vermelha**” de retorno

fim-para

fim-do-procedimento

Eliminando PS

Busca em profundidade p/ digrafos

Versão não recursiva

procedimento $P(v)$

$v \leftarrow$ *topo da pilha*; $PE(v) \leftarrow 1$

se $N_{out}(v)$ *não está vazia* então

Introduzindo a pilha - 1

para todo vértice w em $N_{out}(v)$ faça

remover w de $N_{out}(v)$

se $PE(w) = 0$

então *empilhar w e interromper “para”*

senão se w *está na pilha*

então marcar vw como *aresta “vermelha” de retorno*

fim-para

senão *desempilhar v*

fim-do-procedimento

Busca em profundidade p/ digrafos

Versão não recursiva

enquanto pilha não vazia faça

$v \leftarrow \text{topo da pilha}; PE(v) \leftarrow 1$

se $N_{out}(v)$ não está vazia então

para todo vértice w em $N_{out}(v)$ faça

remover w de $N_{out}(v)$

se $PE(w) = 0$

então empilhar w e interromper “para”

senão se w está na pilha

então marcar vw como aresta “vermelha” de retorno

fim-para

senão desempilhar v

fim-enquanto

Introduzindo a pilha - 2

Busca em profundidade p/ digrafos

Versão não recursiva

enquanto pilha não vazia faça

$v \leftarrow \text{topo da pilha}; PE(v) \leftarrow 1$

se $N_{out}(v)$ não está vazia então

para todo vértice w em $N_{out}(v)$ faça

remover w de $N_{out}(v)$

se $PE(w) = 0$

então empilhar w e interromper “para”

senão se w está na pilha

então marcar vw como aresta “**vermelha**” de retorno

fim-para

senão desempilhar v

fim-enquanto

Código final

Busca em profundidade p/ digrafos

Versão não recursiva

while pilha != []:

$v \leftarrow \text{topo da pilha}; PE(v) \leftarrow 1$

se $N_{out}(v)$ não está vazia então

para todo vértice w em $N_{out}(v)$ faça

remover w de $N_{out}(v)$

se $PE(w) = 0$

então empilhar w e interromper “para”

senão se w está na pilha

então marcar vw como aresta “vermelha” de retorno

fim-para

senão desempilhar v

Passando para Python!

Busca em profundidade p/ digrafos

Versão não recursiva

while pilha != []:

$v \leftarrow \text{topo da pilha}; PE(v) \leftarrow 1$

se $N_{out}(v)$ não está vazia então

para todo vértice w em $N_{out}(v)$ faça

remover w de $N_{out}(v)$

se $PE(w) = 0$

então empilhar w e interromper “para”

senão se w está na pilha

então marcar vw como aresta “vermelha” de retorno

fim-para

senão desempilhar v

Passando para Python!

Busca em profundidade p/ digrafos

Versão não recursiva

while pilha != []:

v = pilha[len(pilha)-1]

PE(v) ← 1

se $N_{out}(v)$ não está vazia então

para todo vértice w em $N_{out}(v)$ faça

remover w de $N_{out}(v)$

se $PE(w) = 0$

então empilhar w e interromper “para”

senão se w está na pilha

então marcar vw como aresta “vermelha” de retorno

fim-para

senão desempilhar v

Passando para Python!

Busca em profundidade p/ digrafos

Versão não recursiva

while pilha != []:

v = pilha[len(pilha)-1]

marca[v] = 1

se $N_{out}(v)$ não está vazia então

para todo vértice w em $N_{out}(v)$ faça

remover w de $N_{out}(v)$

se $PE(w) = 0$

então empilhar w e interromper “para”

senão se w está na pilha

então marcar vw como **aresta “vermelha”** de retorno

fim-para

senão desempilhar v

Passando para Python!

Busca em profundidade p/ digrafos

Versão não recursiva

while pilha != []:

v = pilha[len(pilha)-1]

marca[v] = 1

if n_out[v] != []:

para todo vértice w em $N_{out}(v)$ faça

remover w de $N_{out}(v)$

se $PE(w) = 0$

então empilhar w e interromper “para”

senão se w está na pilha

então marcar vw como aresta “vermelha” de retorno

fim-para

senão desempilhar v

Passando para Python!

Busca em profundidade p/ digrafos

Versão não recursiva

```
while pilha != []:
```

```
    v = pilha[len(pilha)-1]
```

```
    marca[v] = 1
```

```
    if n_out[v] != []:
```

```
        for w in n_out[v]:
```

remover w de $N_{out}(v)$

se $PE(w) = 0$

então empilhar w e interromper “para”

senão se w está na pilha

então marcar vw como aresta “vermelha” de retorno

fim-para

senão desempilhar v

Passando para Python!

Busca em profundidade p/ digrafos

Versão não recursiva

while pilha != []:

v = pilha[len(pilha)-1]

marca[v] = 1

if n_out[v] != []:

for w in n_out[v]:

remover w de $N_{out}(v)$

se $PE(w) = 0$

então empilhar w e interromper “para”

senão se w está na pilha

então marcar vw como aresta “vermelha” de retorno

Passando para Python!

senão desempilhar v

Busca em profundidade p/ digrafos

Versão não recursiva

```
while pilha != []:  
    v = pilha[len(pilha)-1]  
    marca[v] = 1  
    if n_out[v] != []:  
        for w in n_out[v]:  
            del n_out[v][0]  
            se  $PE(w) = 0$   
                então empilhar w e interromper “para”  
  
            senão se w está na pilha  
                então marcar vw como aresta “vermelha” de retorno
```

Passando para Python!

senão *desempilhar v*

Busca em profundidade p/ digrafos

Versão não recursiva

```
while pilha != []:  
    v = pilha[len(pilha)-1]  
    marca[v] = 1  
    if n_out[v] != []:  
        for w in n_out[v]:  
            del n_out[v][0]  
            if marca[w] == 0:  
                então empilhar w e interromper “para”
```

Passando para Python!

senão se *w está na pilha*
 então marcar vw como *aresta “vermelha” de retorno*

senão *desempilhar v*

Busca em profundidade p/ digrafos

Versão não recursiva

```
while pilha != []:  
    v = pilha[len(pilha)-1]  
    marca[v] = 1  
    if n_out[v] != []:  
        for w in n_out[v]:  
            del n_out[v][0]  
            if marca[w] == 0:  
                pilha.append(w)  
                break
```

Passando para Python!

senão se *w está na pilha*

então marcar vw como *aresta “vermelha” de retorno*

senão desempilhar v

Busca em profundidade p/ digrafos

Versão não recursiva

```
while pilha != []:
    v = pilha[len(pilha)-1]
    marca[v] = 1
    if n_out[v] != []:
        for w in n_out[v]:
            del n_out[v][0]
            if marca[w] == 0:
                pilha.append(w)
                break
    elif w in pilha:
```

Passando para Python!

então marcar vw como *aresta “vermelha” de retorno*

senão desempilhar v

Busca em profundidade p/ digrafos

Versão não recursiva

```
while pilha != []:
    v = pilha[len(pilha)-1]
    marca[v] = 1
    if n_out[v] != []:
        for w in n_out[v]:
            del n_out[v][0]
            if marca[w] == 0:
                pilha.append(w)
                break
    elif w in pilha:
        ciclo = True
```

Passando para Python!

senão desempilhar v

Busca em profundidade p/ digrafos

Versão não recursiva

```
while pilha != []:  
    v = pilha[len(pilha)-1]  
    marca[v] = 1  
    if n_out[v] != []:  
        for w in n_out[v]:  
            del n_out[v][0]  
            if marca[w] == 0:  
                pilha.append(w)  
                break  
            elif w in pilha:  
                ciclo = True  
    else:  
        pilha.pop()
```

Passando para Python!

Busca em profundidade p/ digrafos

Versão não recursiva

```
while pilha != []:  
    v = pilha[len(pilha)-1]  
    marca[v] = 1  
    if n_out[v] != []:  
        for w in n_out[v]:  
            del n_out[v][0]  
            if marca[w] == 0:  
                pilha.append(w)  
                break  
            elif w in pilha:  
                ciclo = True  
    else:  
        pilha.pop()
```

OK!

Busca em profundidade p/ digrafos

Versão não recursiva

INICIALIZAÇÕES

```
n,m = input().split()           # ler numero de vertices e arestas
n = int(n)
m = int(m)
n_out = [[] * n for i in range(n)] # definir listas de adjacencia
for j in range(m):             # ler as m arestas do digrafo
    a,b = input().split()      # ler aresta de a para b
    a = int(a)
    b = int(b)
    n_out[a-1].append(b-1)     # b é vizinho de saída de a
marca = n * [0]               # definir o vetor de marcacao
ciclo = False                 # inicializar variavel ciclo
```

Busca em profundidade p/ digrafos

Versão não recursiva

LOOP PRINCIPAL

```
for a in range(n):  
    if marca(a) == 0:  
        pilha == [a]
```

percorrer os vertices
se a ainda não foi atingido na busca
inicializar pilha com o vértice a

```
while pilha != []:  
    v = pilha[len(pilha)-1]  
    marca[v] = 1  
    if n_out[v] != []:  
        for w in n_out[v]:  
            del n_out[v][0]  
            if marca[w] == 0:  
                pilha.append(w)  
                break  
            elif w in pilha:  
                ciclo = True  
    else:  
        pilha.pop()
```

 código já visto!

```
if ciclo:  
    print('SIM')  
else:  
    print('NÃO')
```

 imprimir mensagem